

PIC24 – Project 5

Project 5

Non-jump Instructions	Flags	Jump Instructions
BTG Ws,#bit4	OV	BRA OV,Expr
CLR Wd	C	BRA C,Expr
NEG Ws,Wd	N	BRA N,Expr
SUBB Wb,#lit5,Wd	Z	BRA Z,Expr

First table (OPCODE instructions):

Encoding	2222 3210	1111 9876	1111 5432	1100 1098	7654	3210	Flags
ADD Wb, Ws, Wd	0100	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
SUB Wb, Ws, Wd	0101	0www	wBqq	qddd	dppp	ssss	N, OV, Z, C
AND Wb, Ws, Wd	0110	0www	wBqq	qddd	dppp	ssss	N, Z
IOR Wb, Ws, Wd	0111	0www	wBqq	qddd	dppp	ssss	N, Z
MOV Wns, f	1000	1fff	ffff	ffff	ffff	ssss	none
MOV f, Wnd	1000	0fff	ffff	ffff	ffff	dddd	none
BRA Expr	0011	0111	nnnn	nnnn	nnnn	nnnn	none
BTG Ws,#bit4	1010	0010	bbbb	0B00	0ppp	ssss	none
CLR Wd	1110	1011	0Bqq	qddd	d000	0000	none
NEG Ws,Wd	1110	1010	0Bqq	qddd	dppp	ssss	N, OV, Z, C
SUBB Wb,#lit5,Wd	0101	1www	wBqq	qddd	d11k	kkkk	N, OV, Z, C
BRA OV, Expr	0011	0000	nnnn	nnnn	nnnn	nnnn	none

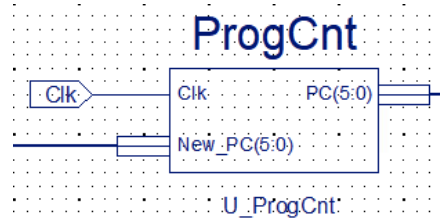
BRA C, Expr	0011	0001	nnnn	nnnn	nnnn	nnnn	none
BRA N, Expr	0011	0011	nnnn	nnnn	nnnn	nnnn	none
BRA Z, Expr	0011	0010	nnnn	nnnn	nnnn	nnnn	none

Second table (signals generated by the control block):

Encoding	OPCODE	CE_NF	CE_OVF	CE_ZF	CE_CF	ALUOP	MemWr	Mem2Reg	RegWr	Branch	RegDest	RegBase
ADD Wb, Ws, Wd	01000XXX	1	1	1	1	000	0	0	1	0	0	0
SUB Wb, Ws, Wd	01010XXX	1	1	1	1	001	0	0	1	0	0	0
AND Wb, Ws, Wd	01100XXX	1	0	1	0	010	0	0	1	0	0	0
IOR Wb, Ws, Wd	01110XXX	1	0	1	0	011	0	0	1	0	0	0
MOV f, Wnd	10000XXX	0	0	0	0	xxx	0	1	1	0	1	X
MOV Wns, f	10001XXX	0	0	0	0	xxx	1	0	0	0	X	X
BRA Expr	00110111	0	0	0	0	xxx	0	0	0	1	X	X
BTG Ws, #bit4	10100010	0	0	0	0	111	0	0	1	0	1	X
CLR Wd	11101011	0	0	0	0	110	0	0	1	0	0	X
NEG Ws, Wd	11101010	1	1	1	1	101	0	0	1	0	0	X
SUBB Wb, #lit5, Wd	01011XXX	1	1	1	1	100	0	0	1	0	0	0
BRA OV, Expr	00110000	0	0	0	0	xxx	0	0	0	1	X	X
BRA C, Expr	00110001	0	0	0	0	xxx	0	0	0	1	X	X
BRA N, Expr	00110011	0	0	0	0	xxx	0	0	0	1	X	X
BRA Z, Expr	00110010	0	0	0	0	xxx	0	0	0	1	X	X

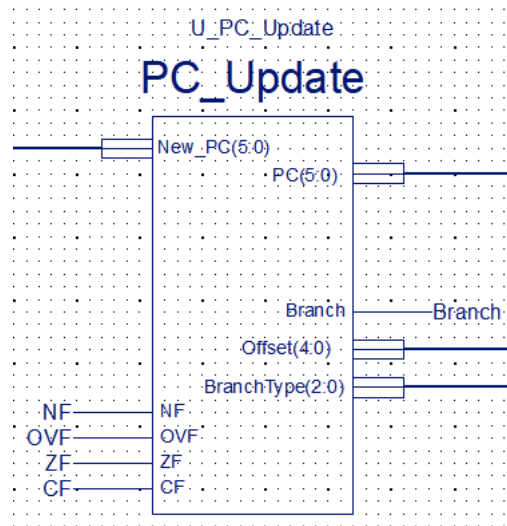


ProgCnt:



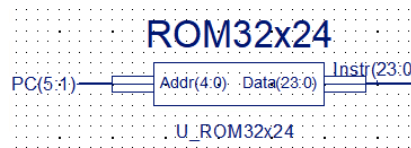
The **ProgCnt** (Program Counter) block updates the program counter (PC) value using the results calculated by the **PC_Update** block. This update is synchronized with the clock signal (Clk). The block is implemented with a 6-bit width, is initially reset to the value "000000", and updates on every rising edge of the clock signal.

PC_Update:



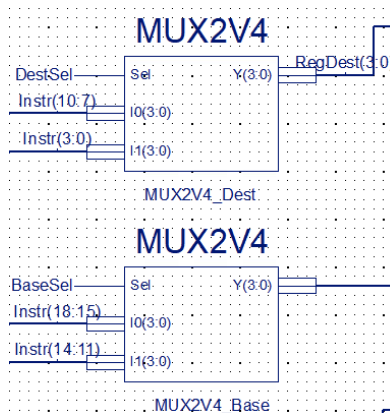
The **PC_Update** block is responsible for calculating the new value of the PC. For standard, non-branching instructions, the PC value is incremented by 2. In the case of branch instructions, the offset given by bits 4:0 of the instruction is shifted left by one position and then added to this increment of 2, so that $\text{New_PC} = \text{PC} + 2 + 2 \times \text{Offset}$. The shift is necessary because the PC counts in bytes while consecutive instructions are two bytes apart. The exact type of branch instruction is identified using bits 18:16 (represented by the BranchType pin) to differentiate between various branch types that share the same opcode.

ROM32X24:



The processor's memory block consists of 32 locations, each capable of storing 24-bit data. These locations store the machine code for each instruction, which is essential for the processor's operation.

MUX2V4:



Two multiplexers of type **MUX2V4** are used in the schematic. Although they are identical blocks, they have different roles: **MUX2V4_Dest** selects the destination register, and **MUX2V4_Base** selects the base register. Both are controlled by signals generated by the Control Unit.

MUX2V4_Dest receives bits 10:7 of the instruction on input I0 and bits 3:0 on input I1, and its selection is made by the RegDest control signal. Its output, RegDest(3:0), is sent to the WrReg input of the register file.

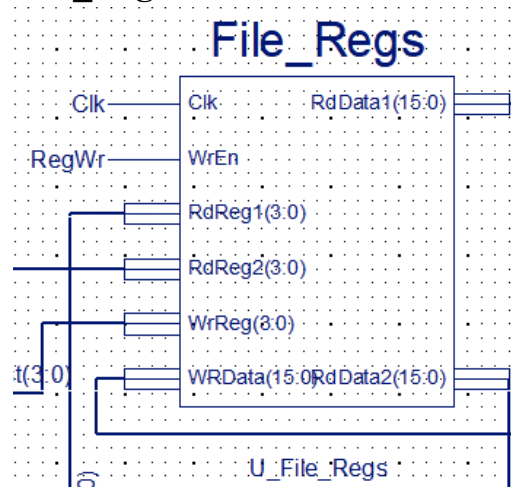
- RegDest is 0, bits 10:7 are selected. These are used by ADD, SUB, AND, IOR, CLR, NEG and SUBB.
- RegDest is 1, bits 3:0 are selected. These are used by MOV f, Wnd and by BTG, which writes the result back into the same register it reads.

MUX2V4_Base receives bits 18:15 on input I0 and bits 14:11 on input I1, and its selection is made by the RegBase control signal. Its output, RegBase(3:0), is sent to the RdReg1 input of the register file.

- RegBase is 0, bits 18:15 are selected. These contain the base register Wb.
- RegBase is 1, bits 14:11 are selected.

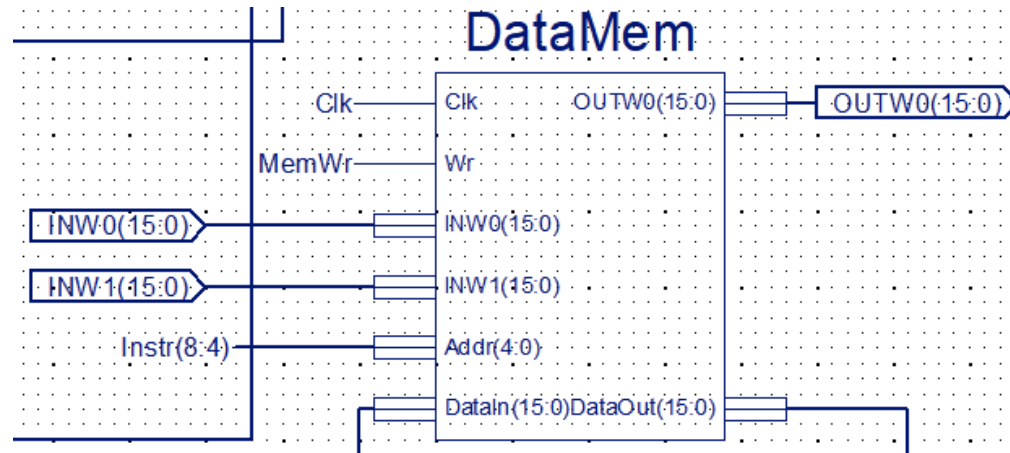
None of the instructions implemented in this project takes its base register from bits 14:11, so the Control Unit drives RegBase to 0 for every opcode. The multiplexer is kept in the schematic so that the datapath remains general and an instruction that uses this field could be added later without modifying the structure.

File_Regs:



This block represents a **Register File**, made up of 16 registers of 16 bits each (W0 - W15), used to temporarily store data during instruction execution. The WrEn (Write Enable) signal, which is equivalent to the RegWr signal generated by the Control Unit, allows data to be written to these registers only when it is active. The RdReg1 and RdReg2 signals specify the addresses of the registers from which data is read, outputting the values through the RdData1 and RdData2 ports, respectively. RdReg2 receives bits 3:0 of the instruction and therefore selects the source register Ws, while RdReg1 receives the output of MUX2V4_Base and therefore selects the base register Wb. Writing is done on the rising edge of the clock, at the address given by WrReg, with the data received on WrData.

DataMem:



The **DataMem** block is a 16x16 RAM (16 locations, 16 bits each) used for reading and writing data.

Data Writing

The **MOV Wns, f** instruction writes data into the RAM. To enable the write operation, the **Wr** signal must be active, and the fourth bit of the address, **Addr(4)**, must be 0. Under these conditions the data from the **DataIn** signal is written to the address specified by **Addr**.

Data Reading

During a read operation, the **DataOut** block provides data based on the value of **Addr(4)**:

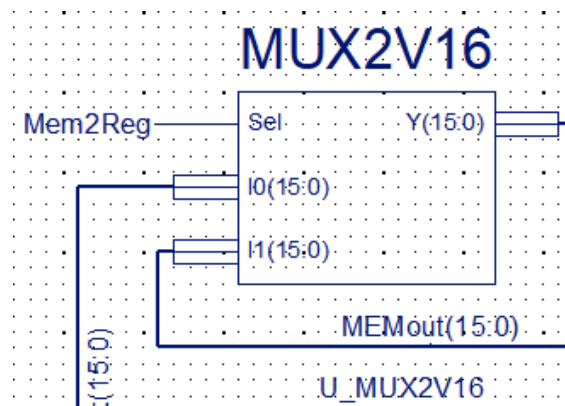
- **Addr(4) = 0**, **DataOut** provides the content from the RAM.

- $\text{Addr}(4) = 1$, DataOut provides the values from the INW0 or INW1 inputs.

The three special locations are INW0 at address 0x1020, INW1 at 0x1022 and OUTW0 at 0x1024. Since $\text{Addr}(4:0)$ is taken from bits 8:4 of the instruction, these become 10000, 10001 and 10010, which is why $\text{Addr}(4) = 1$ identifies a port instead of a RAM location.

The OUTW0 signal transmits data directly from DataIn under specific conditions to ensure fast data access.

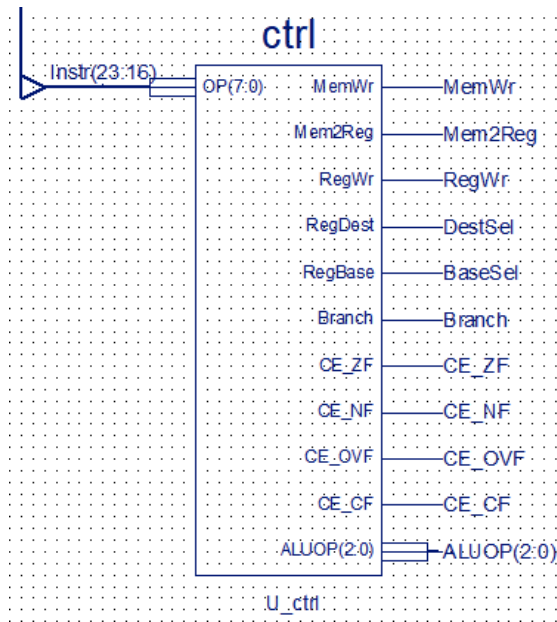
MUX2V16:



The **MUX2V16** multiplexer selects the source of the data to be written into the register. This decision is made based on the **Mem2Reg** control signal, which is generated by the Control Unit.

- $\text{Mem2Reg} = 1$: The data originates from the Data Memory (used for instructions such as MOV f, Wnd).
- $\text{Mem2Reg} = 0$: The data originates from the Arithmetic Logic Unit (ALU).

ctrl:



The **Control** block is responsible for generating the control signals necessary to ensure the correct execution of instructions within the processor. Based on the opcode, this block makes decisions regarding ALU functionality, data routing, and the configuration of the processor's internal states.

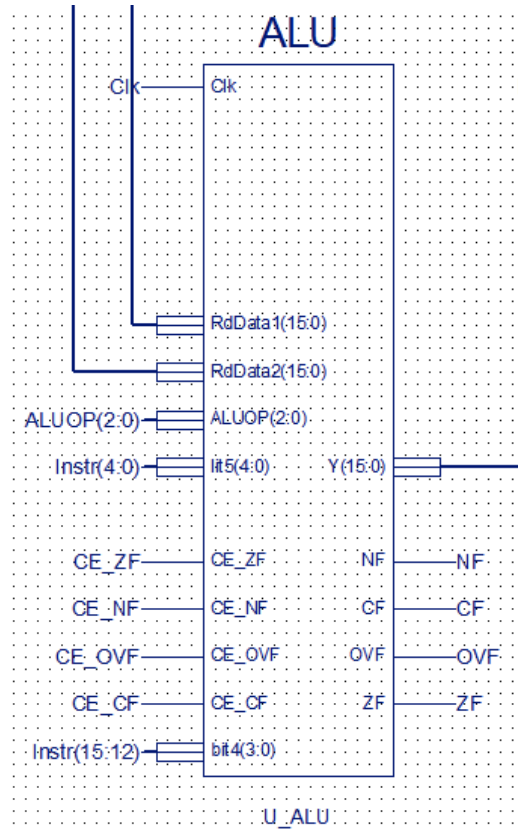
Control Signals

- **MemWr:** Enables writing to the data memory. It is set to 1 only for the MOV Wns, f instruction.
- **Mem2Reg:** Controls data selection in the MUX2V16 multiplexer. If the value is 1, the data to be written into the register originates from memory, otherwise it originates from the ALU output.

- **RegWr:** Enables register writing. It is set to 1 for instructions that modify registers (ADD, SUB, AND, IOR, MOV f Wnd, BTG, CLR, NEG and SUBB) and 0 for the branch instructions and for MOV Wns, f.
- **RegDest:** Controls the register destination in the MUX2V4_Dest multiplexer. It is 0 when the destination register is identified by bits 10:7 of the instruction and 1 when it is identified by bits 3:0.
- **RegBase:** Controls the base register selection in the MUX2V4_Base multiplexer. It is 0 when the base register is identified by bits 18:15 and 1 when it is identified by bits 14:11. It is 0 for the instructions that actually read a base register (ADD, SUB, AND, IOR, SUBB) and don't-care for the rest, since they do not use the output of this multiplexer. In the implementation the signal is driven to 0 unconditionally, which is one of the permitted values in both cases.
- **Branch:** A signal for branch-type instructions, used to update the Program Counter.
- **CE_OVF:** Enables the Overflow Flag in the ALU for the arithmetic instructions ADD, SUB, NEG and SUBB.
- **CE_CF:** Enables the Carry Flag for the arithmetic instructions ADD, SUB, NEG and SUBB.
- **CE_NF:** Enables the Negative Flag for the logical and arithmetic instructions ADD, SUB, AND, IOR, NEG and SUBB.
- **CE_ZF:** Enables the Zero Flag for the logical and arithmetic instructions ADD, SUB, AND, IOR, NEG and SUBB.
- **ALUOP (3 bits):** Encodes the operation sent to the ALU based on the opcode, according to the second table. For the instructions that do not use the ALU (MOV and the branch instructions) the value is a don't care and is marked with "xxx" in the second table.

The Control Unit decodes eight bits, $OP(7:0) = Instr(23:16)$. This is necessary because NEG and CLR have identical bits 23:19 (11101) and can only be distinguished by bit 16: the encoding 11101010 corresponds to NEG and 11101011 corresponds to CLR.

ALU:



The Arithmetic Logic Unit (ALU) is responsible for all arithmetic and logical operations. The ALUOP signal, received from the Control Unit, determines which operation to execute.

The ALU operates synchronously with the clock signal (Clk). Its primary inputs are RdData2, which contains the value of the source register Ws, and RdData1, which contains the value of the base register Wb, while the operation result is sent to output Y. In

addition, the ALU receives the 5-bit literal $\text{lit5}(4:0)$ from bits 4:0 of the instruction, used by SUBB, and the bit position $\text{bit4}(3:0)$ from bits 15:12, used by BTG.

The subtractions are performed using the explicit two's complement form $(\text{'0'} \& A) + (\text{'0'} \& \text{not}(B)) + 1$, on 17 bits, where A is RdData1 (the base register Wb) and B is RdData2 (the source register Ws), so that bit 16 of the result gives directly the value of the Carry flag. NEG uses the same form with the constant 0 in place of A, and SUBB replaces B with the literal lit5 and the constant 1 with the stored value of the Carry flag.

Additionally, the ALU includes enable signals for calculating specific status flags: CE_NF (for the negative flag), CE_OVF (for overflow), CE_ZF (for zero), and CE_CF (for carry).

Status Flags:

- N (Negative): Indicates if the result of the operation is negative.
- OV (Overflow): Indicates that the result exceeds the maximum representable value.
- Z (Zero): Indicates if the result of the operation is zero.
- C (Carry): Indicates a carry out from arithmetic operations.

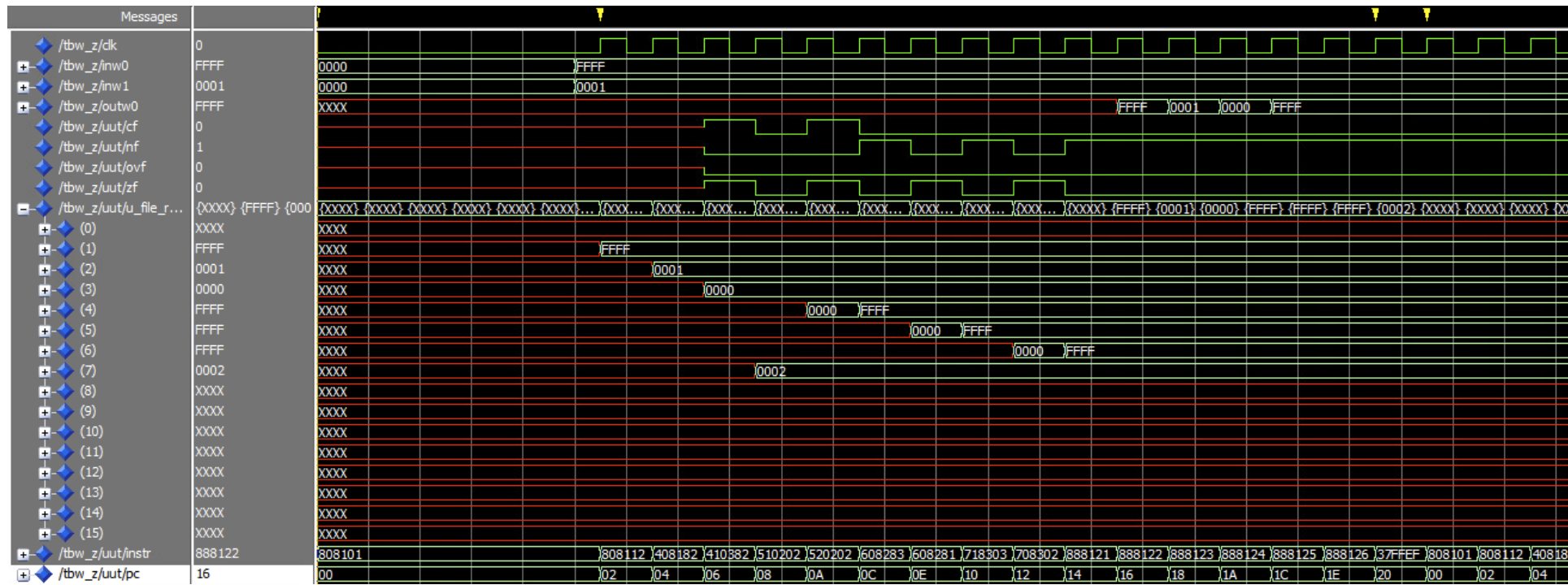
Implemented Instructions:

- **ADD Wb, Ws, Wd** Adds the source register to the base register, $Wd = Wb + Ws$. The addition is performed on 17 bits, in the form $(\text{'0'} \& Wb) + (\text{'0'} \& Ws)$, so that bit 16 of the result gives directly the value of the Carry flag. Affected flags: N, OV, Z, C.
- **SUB Wb, Ws, Wd** Subtracts the source register from the base register, $Wd = Wb - Ws$. It is implemented using the explicit two's complement form $(\text{'0'} \& Wb) + (\text{'0'} \& \text{not}(Ws)) + 1$, on 17 bits, where bit 16 of the result gives the value of the Carry flag: $C = 1$ means that no borrow occurred. Affected flags: N, OV, Z, C.

- **AND Wb, Ws, Wd** Performs the bitwise logical AND between the base register and the source register, $Wd = Wb \text{ AND } Ws$. Being a logical operation, it does not generate a carry or an overflow, so CE_CF and CE_OVF are 0 and the C and OV flags keep their previous values. Affected flags: N, Z.
- **IOR Wb, Ws, Wd** Performs the bitwise logical OR between the base register and the source register, $Wd = Wb \text{ OR } Ws$. As in the case of AND, the C and OV flags are not modified, their enable signals being 0. Affected flags: N, Z.
- **MOV f, Wnd** Reads a word from the data memory, or from one of the input ports INW0 and INW1, and writes it into the destination register. It does not use the ALU. Affected flags: none.
- **MOV Wns, f** Writes the contents of the source register into the data memory, or into the output port OUTW0. Affected flags: none.
- **BRA Expr** Unconditional jump. The new value of the PC is obtained by adding the offset from bits 4:0 to the incremented value of the PC. Affected flags: none.
- **BRA OV/C/N/Z, Expr** Conditional jumps. The jump is taken only if the corresponding flag has the value 1, otherwise the execution continues sequentially. The branch type is identified by bits 18:16 of the instruction. Affected flags: none.
- **BTG Ws, #bit4** Toggles the bit of the register Ws whose position is given by the bbbb field, taken from bits 15:12 of the instruction. It is implemented in the ALU as an XOR between RdData2 and a mask that contains a single '1' on the indicated position. The result is written back into the same register Ws, which is why RegDest is 1 for this instruction. Affected flags: none.
- **CLR Wd** Clears the destination register, writing the value 0x0000 into it. The ALU forces its output to zero without using any of the operands. Affected flags: none, so all four CE signals are 0.
- **NEG Ws, Wd** Computes the two's complement of the source register, $Wd = \text{not}(Ws) + 1$, which is equivalent to the subtraction $0 - Ws$. Affected flags: N, OV, Z, C.
- **SUBB Wb, #lit5, Wd** Subtracts from the base register the 5-bit literal and the borrow, $Wd = Wb - \text{lit5} - (\text{not } C)$. It is implemented as $Wb + \text{not}(\text{lit5}) + C$, where C is the current value of the Carry flag. Z behaves differently from the other flags: SUBB can only clear it, never set it, so a zero result leaves Z unchanged if it was already 0. This keeps Z valid across a multi-word subtraction. Affected flags: N, OV, Z, C.

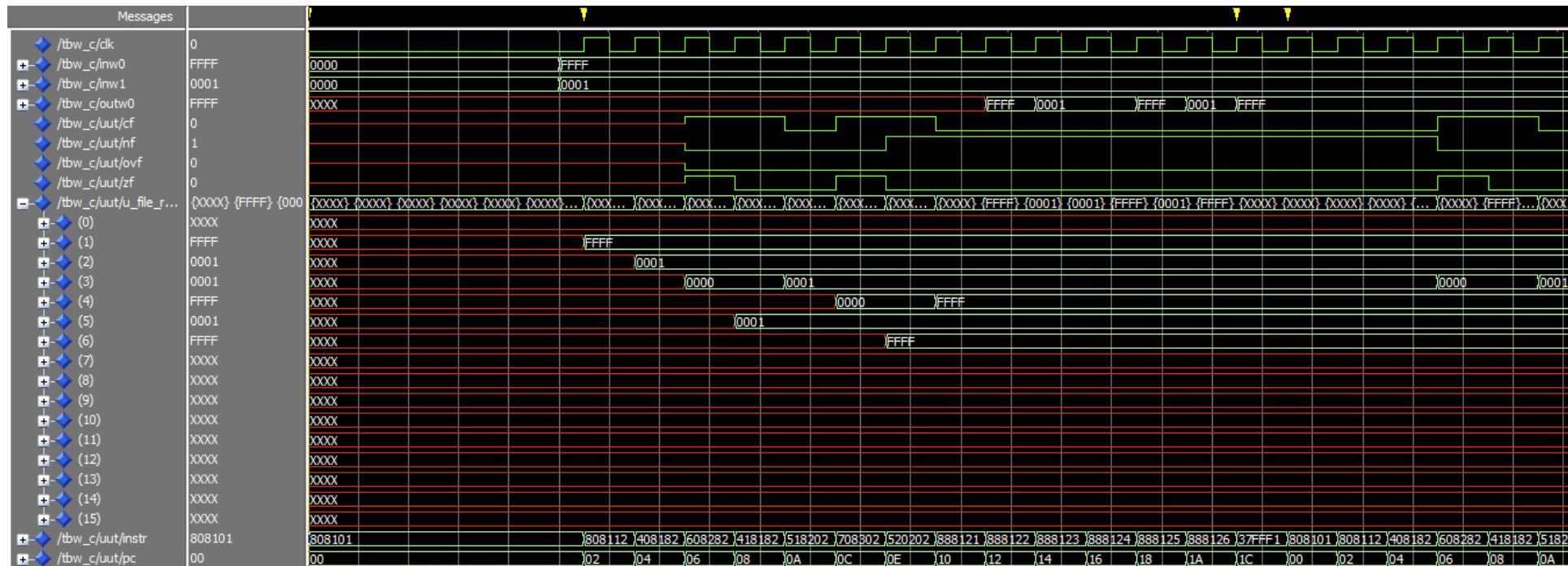
Test All sequence:

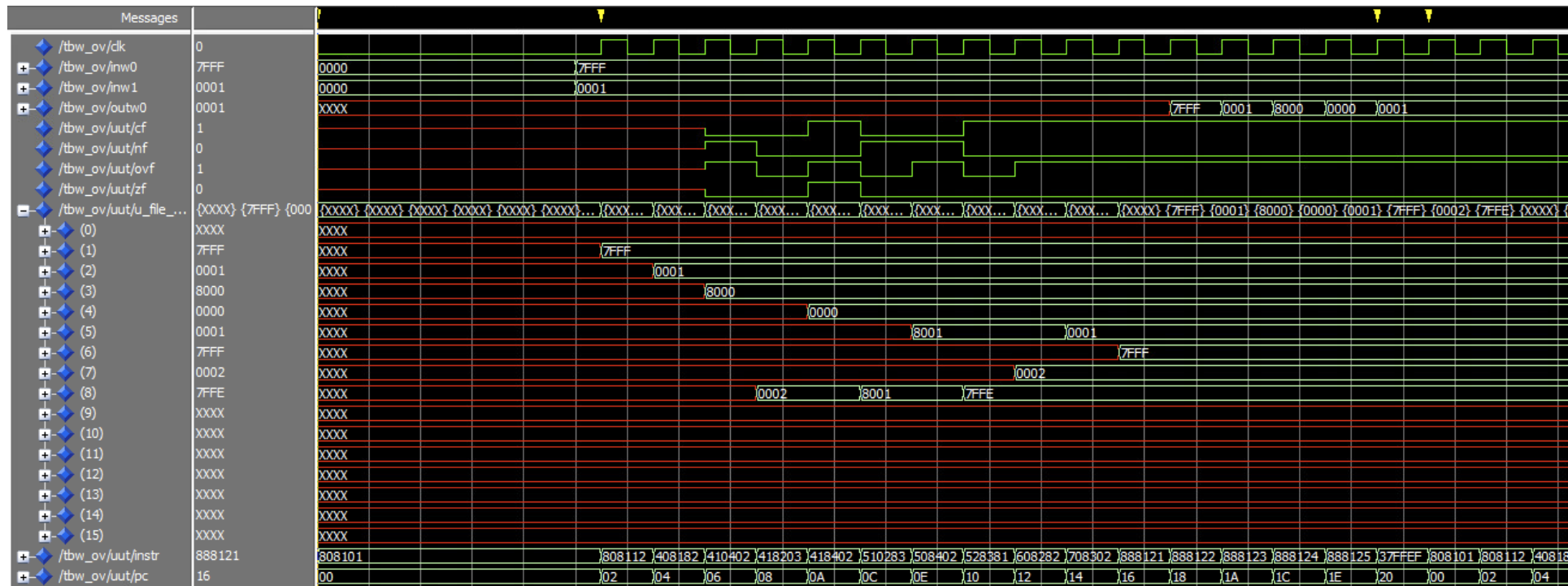




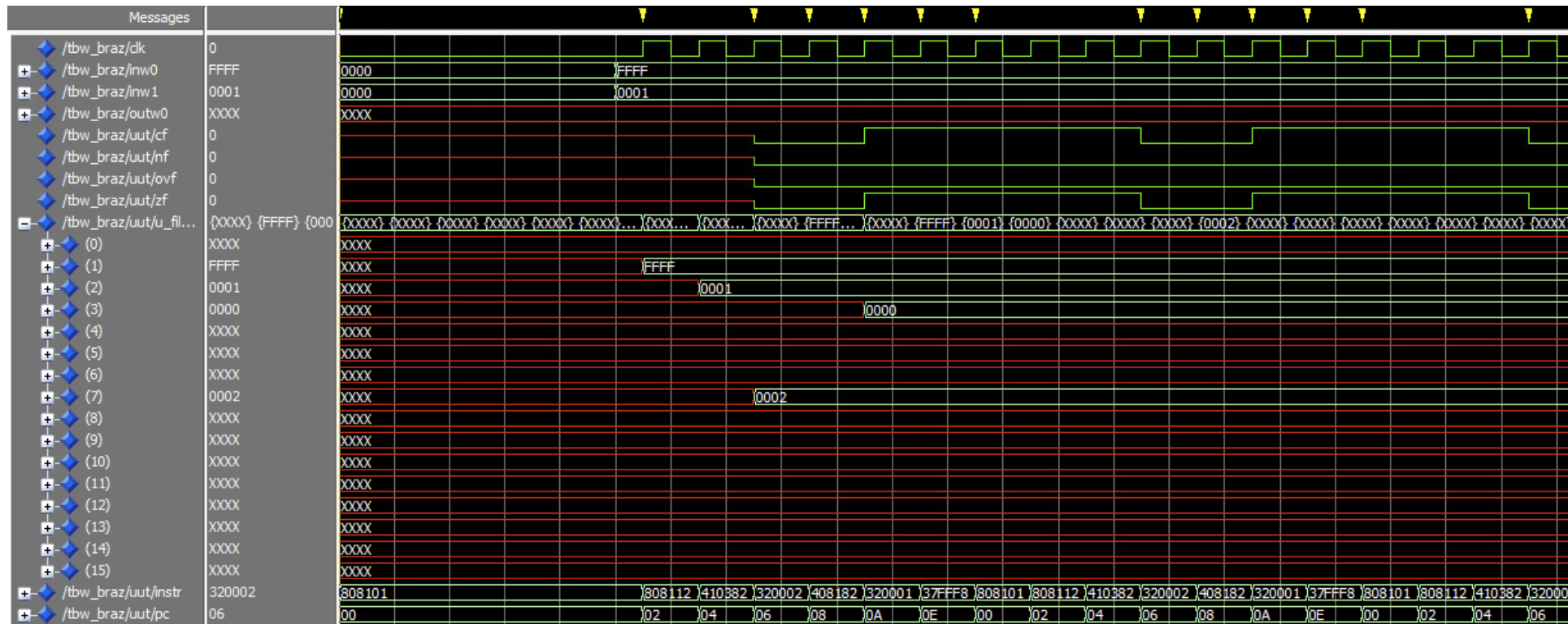
[illegible]

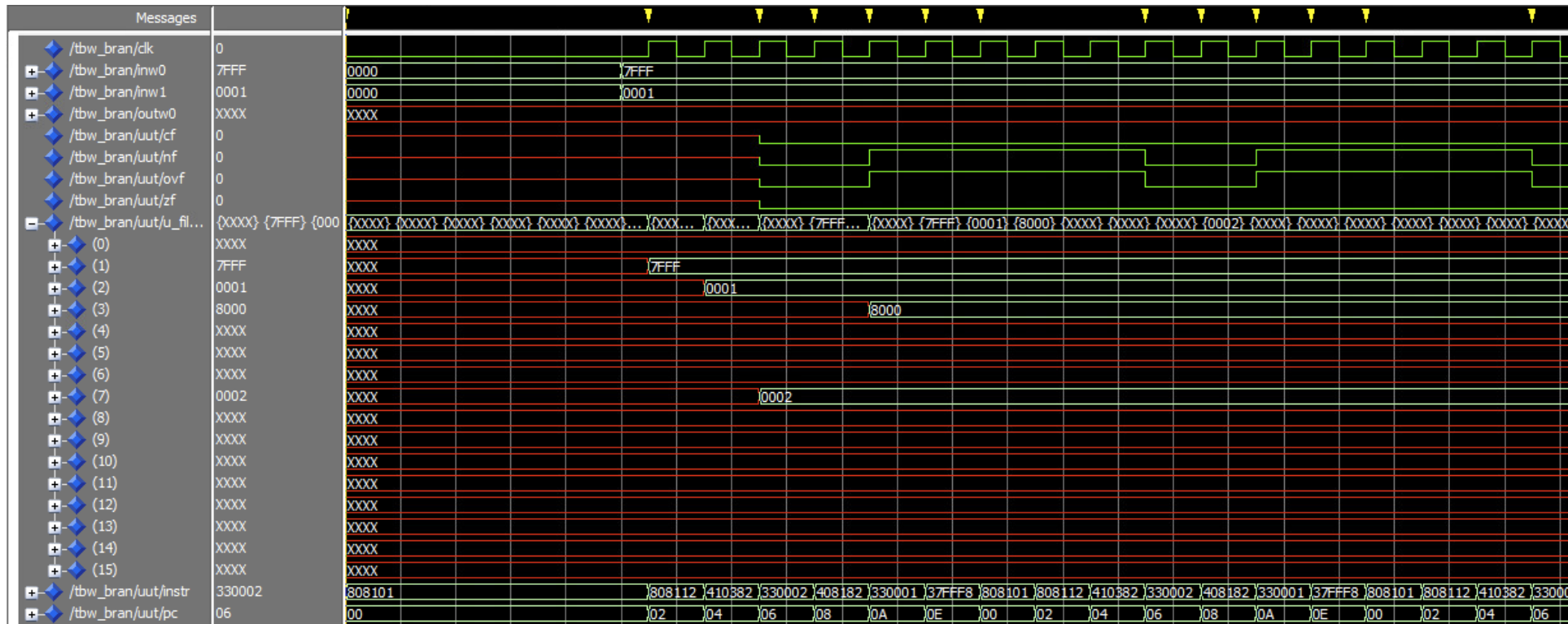
Test C :

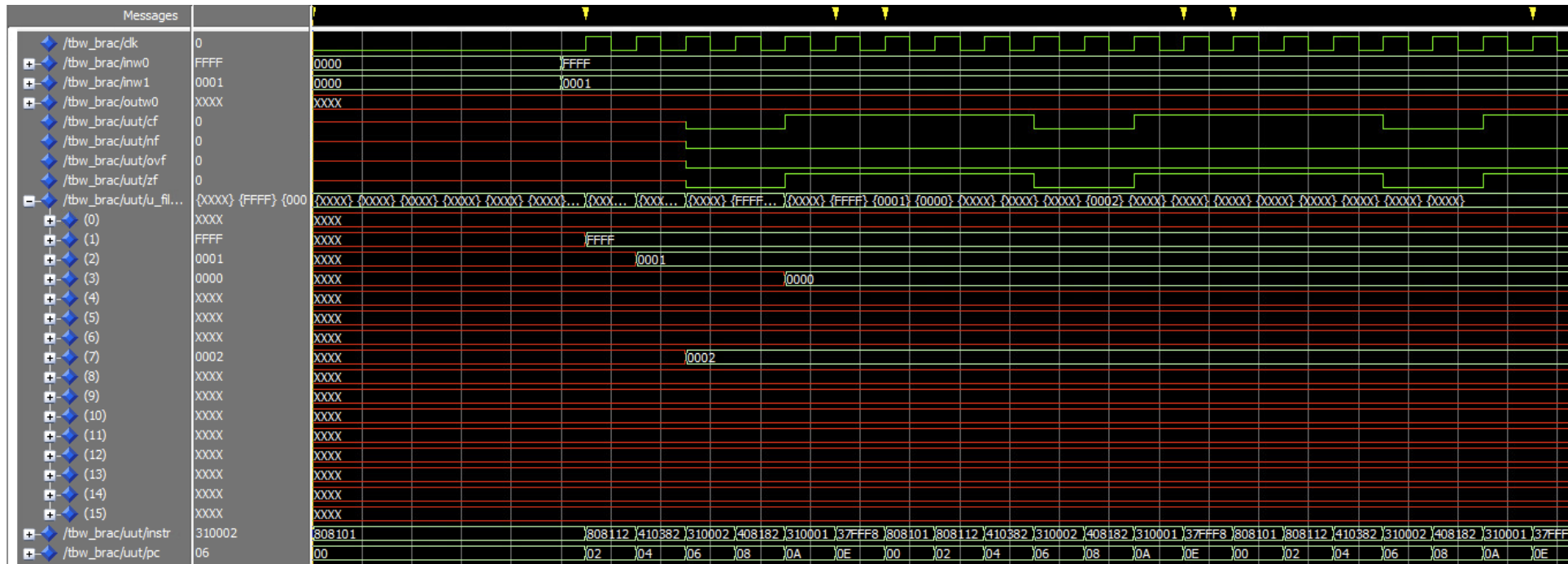




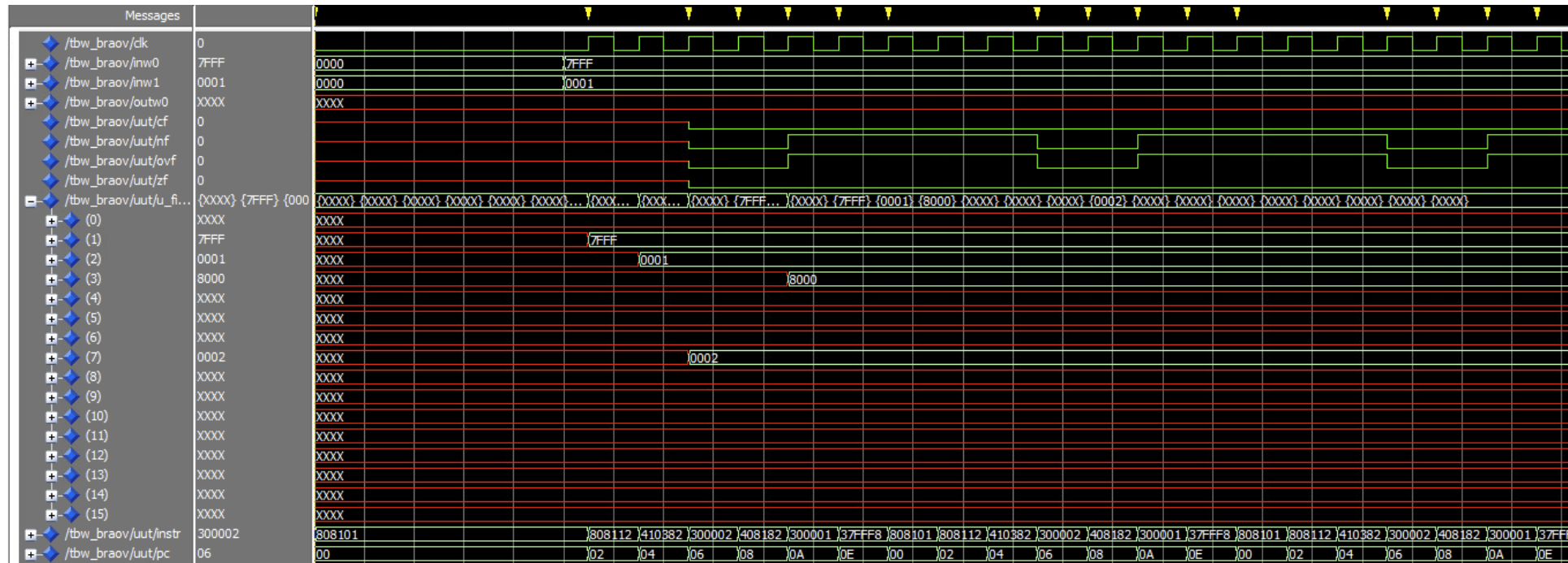
Test BRA Z:



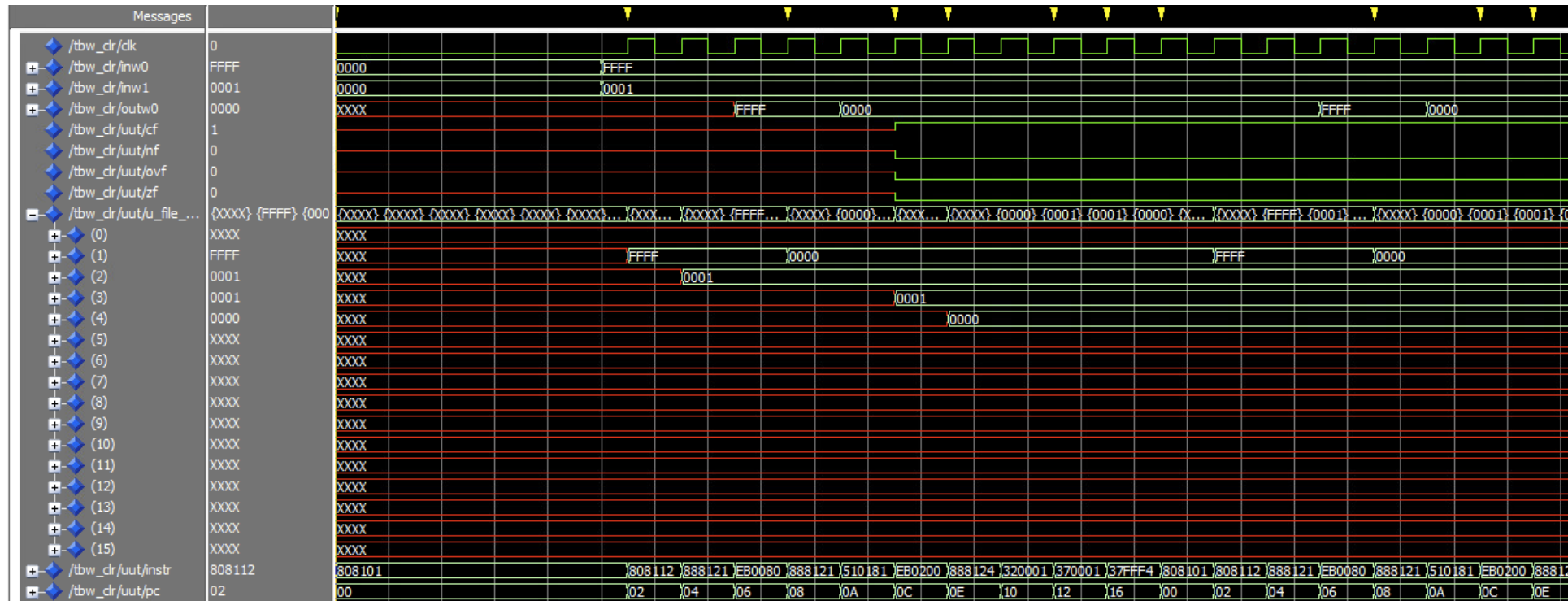


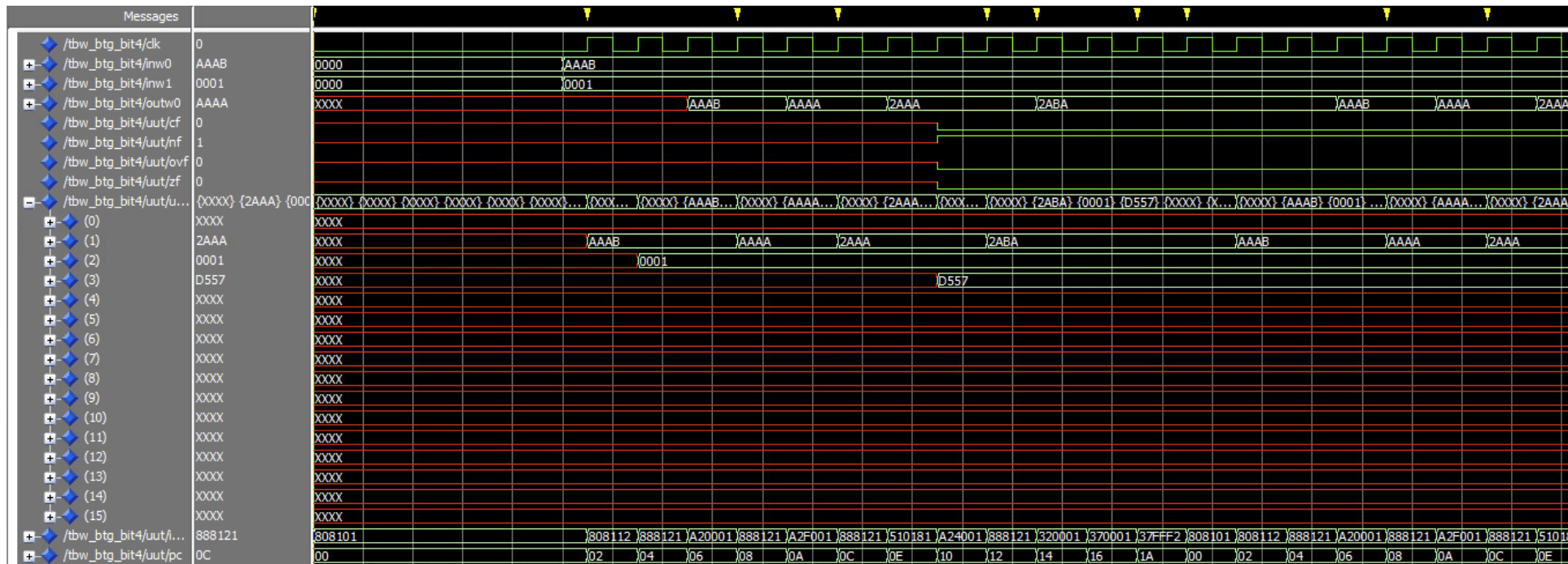


Test BRA OV:

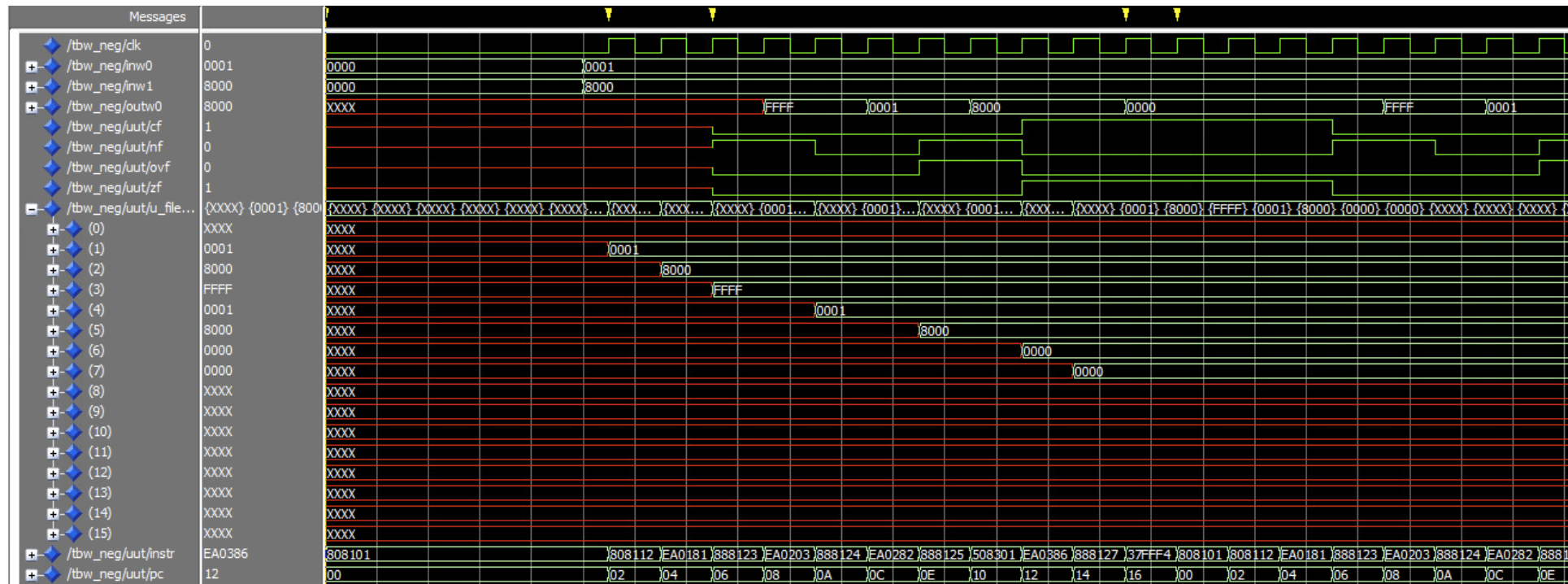


Test CLR Wd:





Test NEG Ws,Wd:



Messages			
+	/tbw_subb_lit5/dk	0	
+	/tbw_subb_lit5/inw0	0010	
+	/tbw_subb_lit5/inw1	8000	
+	/tbw_subb_lit5/outw0	0000	
+	/tbw_subb_lit5/uut/cf	1	
+	/tbw_subb_lit5/uut/nf	0	
+	/tbw_subb_lit5/uut/...	0	
+	/tbw_subb_lit5/uut/zf	0	
+	/tbw_subb_lit5/uut/...	{0000} {0010} {8000}	
+	(0)	XXXX	
+	(1)	0010	
+	(2)	8000	
+	(3)	0000	
+	(4)	0000	
+	(5)	000F	
+	(6)	0000	
+	(7)	7FFF	
+	(8)	XXXX	
+	(9)	XXXX	
+	(10)	XXXX	
+	(11)	XXXX	
+	(12)	XXXX	
+	(13)	XXXX	
+	(14)	XXXX	
+	(15)	XXXX	
+	/tbw_subb_lit5/uut/i...	888125	
+	/tbw_subb_lit5/uut/pc	0C	